

a) Operations

→ Append (10)

→ Append (20)

→ Append (15)

Linked List

After Append (10):

10 → NULL

Current = 10

After append (20):

10 → 20 → NULL

Current = 20

After Append (15):

10 → 20 → 15 → NULL

Current = 15.

Final List

10 → 20 → 15 → NULL

Explanation -

The append (·) operation adds each new element at the end of the linked list. Therefore the final order is 10 → 20 → 15.

b) Operations

- * Append (10)
- * Append (20)
- * Append (15)
- * Move to Start()
- * Insert (39)
- * Next ()
- * Insert (12)

Linked list :

After Append :

$10 \rightarrow 20 \rightarrow 15$

After Move to Start

Current = 10

After Insert (39):

$39 \rightarrow 10 \rightarrow 20 \rightarrow 15$

After Next ():

Current = 10

After insert (12):

$39 \rightarrow 12 \rightarrow 10 \rightarrow 20 \rightarrow 15$

Final List:

$39 \rightarrow 12 \rightarrow 10 \rightarrow 20 \rightarrow 15 \rightarrow \text{NULL}$

Explanation -

Move to Start () moves the current pointer to the beginning.
Insert () adds a new node before the current position,
while next () moves the pointer to the next node.

Binary search

Given array

66, 111, 212, 323, 432, 543, 643, 649, 777, 888

Case 1:

Key = 111

Step	Low	High	Mid	Value	Result
1	0	9	4	432	Search left
2	0	3	1	111	found

∴ Element 111 is found at position 2 (index = 1)

Case 2: key = 88.

Step	Low	High	Mid	Value	Result
1	0	9	4	432	Search right
2	5	9	7	649	Search right
3	8	9	8	777	Search right
4	9	9	9	888	Found.

∴ Element 88 is found at position 10 (index = 9)

Case 3: key = 999.

Step	low	High	Mid	Value	Result
1	0	9	4	432	Search right
2	5	9	7	649	Search right
3	8	9	8	777	Search right
4	9	9	9	888	Search right
5	Low > High	-	-	-	Not found.

∴ Element 999 is not present in the array.

Analysis

- Binary search works ~~sorted~~ only on sorted arrays
- Time complexity: $O(\log n)$
- Binary Search is faster than Linear Search for large datasets.

3) Stack Operations

Stack size = 5

Operation

Push (45)

Push (22)

push (11)

Push (77)

pop()

push (44)

push (33)

pop()

pop()

Stack (Bottom → Top)

45

45, 22

45, 22, 11

45, 22, 11, 77

45, 22, 11

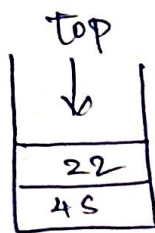
45, 22, 11, 44

45, 22, 11, 44, 33

45, 22, 11, 44

45, 22, 11

Final Stack



Peak element

→ Top element = 22

→ Index position = 1 (0-based indexing)

Explanation

The stack follows the LIFO principle. Elements are inserted using Push and removed using pop. After performing all the operation, the remaining elements are 45 & 22, with 22 at the top of the stack.

Time Complexity

→ Push = $O(1)$

→ Pop = $O(1)$

4. Convert the expression step by step

G_n ,

$((a+b) + c * (d+e) + f) * g + h)$

Step	Scan Symbol	Stack	Postfix Output
1	(	(	
2	(	((	
3	a	((	a
4	+	((+	a
5	b	((+	ab
6	)	(	ab+
7	+	(+	ab+
8	C	(+	ab+C
9	*	(+*	ab+c
10	C	(+*	ab+c
11	d	(+*C	ab+c
12	+	(+*C	ab+c
13	e	(+*C	ab+c
14	)	(+*C	ab+cde
15	+	(+*C	ab+cde+
16	f	(+*	ab+cde+*
17	)	(+	ab+cde+*+f
18	*	(+	ab+cde+*+f+
19	(	(+	ab+cde+*+f+
20	g	NULL	ab+cde+*+f+g
21	+	+	ab+cde+*+f+g
22	n	+	ab+cde+*+f+g
23	)	+	ab+cde+*+f+g
24	h	+	ab+cde+*+f+g
25	end	NULL	ab+cde+*+f+g+h

final postfix

$a b + c d e + * + f + g h + *$

b) Prefix Conversion

Step	Operation	Result
1	Original Expression	$(a + b) + c * (d + e) + f) * (g + h)$
2	Reverse Expression	$(h + g) * (f + (c + d) * c + (b + a))$
3	Convert to Postfix	$+ h g + f * + e d c + b a + *$
4	Reverse Postfix	$* + + + a b * c + d e f + g h$

Final Prefix

$* + + + a b * c + d e f + g h$

Final answers

Expression Type	Result
Prefix	$* + + + a b * c + d e f + g h$
Postfix	$a b + c d e + * + f + g h + *$

5. Priority Queue Vs Simple Queue

Criteria	Simple Queue	Priority Queue
Order of Service	Customer are served in the order the arrive (FIFO)	Customers with higher priority are served first.

Criteria	Simple Queue	Priority Queue
Fairness	High, because everyone waits in arrival order	Lower because normal customers vary wait longer.
Waiting Time	Equal & predictable for all customers.	VIP waiting time is reduced, but others may increase.
Complexity	Enqueue - $O(1)$, Dequeue - $O(1)$	Insert - $O(\log n)$ Delete - $O(\log n)$

Comparison

Simple Queue	Priority Queue
Follows FIFO principle	Follows priority-based service
Fair to all customers	Gives preference to VIP customers
Easy to implement	More efficient for urgent requests but slightly complex.